Kitchen Kompanion design rationale

This report describes the current medium-fidelity Kitchen Kompanion prototype, why its screens and task paths support household food management, and where the design still needs work. The implemented areas are Kitchen inventory, Shopping, and Recipes. Preferences is an unfinished feature with a clear status message and a return action. The report uses the actual source and captured task flows; it does not assert that unrecorded Phase 1 feedback or formal user testing took place.

The app starts on My Kitchen because viewing available food is a useful daily entry point. The screen presents the inventory immediately instead of requiring a visit through a navigation-only home screen. This supports recognition of food already in the household and makes location and expiration information available before planning meals or shopping. The initial nine items provide varied storage locations, owners, units, and expiration intervals.

The bottom navigation remains in the same position across the three implemented feature areas. Text labels accompany the icons so that users do not have to infer the meaning of an image alone. The active tab is visually marked. Keeping the navigation available makes it possible to move from a food record to a recipe or shopping list without remembering a separate path back to a home menu. The cold-start screen and navigation are horizontal design decisions because they determine how the overall app is organized.

Kitchen uses a compact two-column card layout. Each card groups the item name, storage location, ownership, expiration, quantity, and Edit action. These attributes answer different household questions: where food is stored, who owns it, whether it is time-sensitive, and how much remains. Location filters, a search field, and sorting controls reduce the work of scanning the entire collection. The visible count is calculated from the current results, rather than remaining a fixed sample number.

Shopping uses a list rather than copying the Kitchen card grid. A shopping task emphasizes reading an item name, checking it off, and keeping a quantity or note visible while moving through the list. The larger row text and purchase controls reflect that different use. Related actions sit around a scrolling list so that the main task does not depend on finding a control at the bottom of a long page. This is a deliberate distinction in information hierarchy, although the difference in typography across tabs still needs a consistency review.

Horizontal design continued

Recipes shows meal names, brief descriptions, cooking times, serving counts, and an ingredient-availability summary. The browser can be searched by recipe name or ingredient text and filtered to Can Make or Use Expiring. Detail screens provide the full ingredient list and instructions. This keeps long recipe instructions out of the initial browsing view while making them accessible once a user chooses a meal. Counts and ingredient labels are derived from current Kitchen data.

Green emphasizes primary actions and available food. Amber marks expiration warnings and missing ingredients. Color is accompanied by words such as Missing or In Kitchen, so a user can interpret the status without relying on hue alone. Buttons, borders, spacing, and visible labels group related controls. The design uses consistency, visual hierarchy, recognition rather than recall, and visibility of system status as organizing principles; these are rationale for the current choices rather than evidence that a usability study has confirmed them.

Search and filtering should preserve a clear explanation of what a user is seeing. Recipe cards report how many ingredients are available, while an empty filter result says that no recipes were found. Preferences now identifies itself as not implemented instead of showing a blank area. It offers Back to Kitchen and remains reachable through navigation, but is excluded from the supported task count. This conveys the remaining horizontal scope without presenting it as completed functionality.

The current device frame is 640 by 960 CSS pixels. Inventory editing uses a centered dialog and recipe authoring uses its original scrolling form. The added keyboard placeholders and layout reservations in these two areas were removed at the requested prototype scope. Shopping retains its preexisting keyboard stand-in. Touch-keyboard behavior is not modeled consistently across this build, so the screenshots should be read as desktop prototype views.

Remaining horizontal improvements include checking every font size and contrast against the separate client specifications, reconciling Shopping typography with the other tabs, and inspecting keyboard placement on every input path. Fixed-height layouts need testing with more items and longer labels. No compliance claim is made for an unseen hardware specification. Screenshot pages H1 and H2 show the current top-level screens; task screenshots V1 through V4 show implemented paths. The images are separate from these two pages of horizontal discussion.

Vertical design inventory and integration

Inventory editing starts from the food record that needs correction. Selecting Edit opens the existing food-entry form with the record's saved values. The user can change name, quantity, unit, storage location, ownership, and days until expiration. Save changes updates the same record identifier instead of creating another item. Cancel, close, and Escape discard changes. After saving, the list is filtered and sorted again and a status message identifies the updated food.

Reusing the add-food form supports consistency between creating a record and maintaining it. Prefilled values reduce recall and typing, while an explicit save action separates inspection from commitment. Blank names and invalid numeric inputs are rejected by form validation. Keyboard focus starts in the dialog, stays within it while tabbing, and returns to the triggering control after closing. A fallback return target is used when an edit makes the original card disappear from the active location filter.

Food transferred from Shopping can carry an actual expiration date. When its days-until-expiration value is edited, the stored date is updated as well. This prevents an edited value from being immediately replaced by the previous date on the next read. Inventory quantities support fractional amounts, and transferred units remain available in the editor. These details matter because the prototype connects forms across feature sets rather than demonstrating each tab with isolated static values.

Recipes now calls the same inventory reader used by Kitchen. The former recipe-specific reader was absent, which made all food appear unavailable. The expiration comparison now reads daysUntilExpiration, matching the field used by Kitchen. Inventory saves emit a change event that refreshes recipe results, and opening Recipes refreshes the displayed availability. If a recipe detail is already open, its ingredient statuses are updated as well.

The demo uses explicit ingredient aliases, including Eggs and Large eggs, Spinach and Fresh spinach, and Cheese and Cheddar cheese. Names are normalized for case and repeated whitespace. The code deliberately avoids arbitrary substring matching: Peanut butter must not satisfy Butter. A positive quantity and a nonnegative expiration interval are required before a food is treated as available. This makes the current data useful while keeping the matching rule understandable and testable.

The matching model is still limited. It treats ingredient presence as sufficient, even if the available amount is smaller than a recipe would need. Ingredients are stored as names rather than measured quantities. The serving value is recorded and displayed but does not scale ingredient amounts. Dietary restrictions, allergy suitability, ownership permissions, and alternative ingredients are not inferred. A later design should make these constraints explicit before claiming that a meal is suitable or fully provisioned.

Vertical design recipe task paths

Recipe search combines user-entered text with the selected category. A spinach search produces a recipe whose ingredient list can be inspected in detail. Use Expiring includes a recipe when at least one matching food is present, has a positive quantity, and expires between today and three days from today. Expired food is excluded from recipe availability. The filter therefore supports a goal of using food soon while preserving a visible distinction between available and missing ingredients.

Editing spinach from one day until expiration to ten days updates the recommendation results. Removing the quantity or making the item expired also changes its availability. This connection is important for task-centered evaluation: a reviewer can modify Kitchen and observe a consequence in Recipes. The filter is not merely a hard-coded link to a particular recipe screenshot. It uses deterministic local data and explicit aliases to demonstrate the interaction offline.

Authoring a recipe requires a name, preparation time, serving count, description, ingredients, and ordered instructions. The form stores each nonempty ingredient and instruction line as a separate entry. Saving returns to browsing, where the new recipe can be opened on a detail screen. A reviewer can then revise the instructions and inspect the saved result. Persistence in local storage allows this task to be resumed after reloading the page.

The proposed deep authoring task includes creation, inspection, and revision, because that path tests whether detailed information survives transitions between distinct views. The depth label is a proposed assessment, not a guarantee of credit. A brief add-only interaction could be judged moderate. The task guide provides exact text and a specific revision so that another team can repeat the same path and compare the observed behavior with its expected outcome.

The recipe form uses its original scrolling layout for longer ingredients and instructions. Save and Cancel follow the fields in the form. The added reserved keyboard region and special field-only scrolling were removed at the requested prototype scope. It remains necessary to evaluate discoverability of scrolling and comfort of text entry with actual users; an automated browser can verify values and transitions but cannot establish those outcomes.

Feedback and recovery are particularly important during data entry. Inventory cancellation preserves the last saved item, recipe deletion asks for confirmation, and a canceled recipe form returns to the browser. Remaining recipe improvements include field-specific messages rather than a generic alert for invalid content, stronger storage-error feedback, clearer indication of unsaved edits, and focus restoration between detail and browser views. These are recorded as limitations rather than counted as implemented tasks.

Vertical design shopping and evaluation

Shopping separates marking a purchase from adding food to Kitchen. A checkmark changes the purchased state and summary count but does not silently assume a storage location or expiration date. Finish shopping opens a review overview. Each purchased item receives a review form containing name, quantity, unit, storage location, and a date. The next action remains unavailable until the purchased entries have valid reviews.

A separate confirmation view shows the reviewed batches before commitment. Unpurchased entries remain on the shopping list. Confirming adds each purchased entry as its own Kitchen batch and removes the transferred shopping entries. Treating purchases as separate batches preserves different expiration dates and avoids silently merging fresh groceries into an older food record. A completed screen reports the result and offers View Kitchen.

This is the strongest deep task in the current prototype: the reviewer moves through an overview, individual review forms, confirmation, completion, and inventory. Multiple fields have consequences in a second feature set. The staged flow supports error prevention by placing inspection before commitment. A local transfer journal uses stable batch identifiers so an interrupted confirmed transfer can be retried without intentionally adding the same batches again. The reviewed test also reloads the completed state and checks that the count has not increased a second time.

Saved lists preserve a named snapshot of shopping items. The user saves the active list, removes an item from the active list, then previews the snapshot to select what to reuse. Existing item names are disabled and explained as already on the list. Reused entries are added unchecked; the snapshot does not carry purchase checkmarks forward. This avoids a routine shopping template accidentally appearing already purchased. The selective reuse path is counted conservatively as moderate.

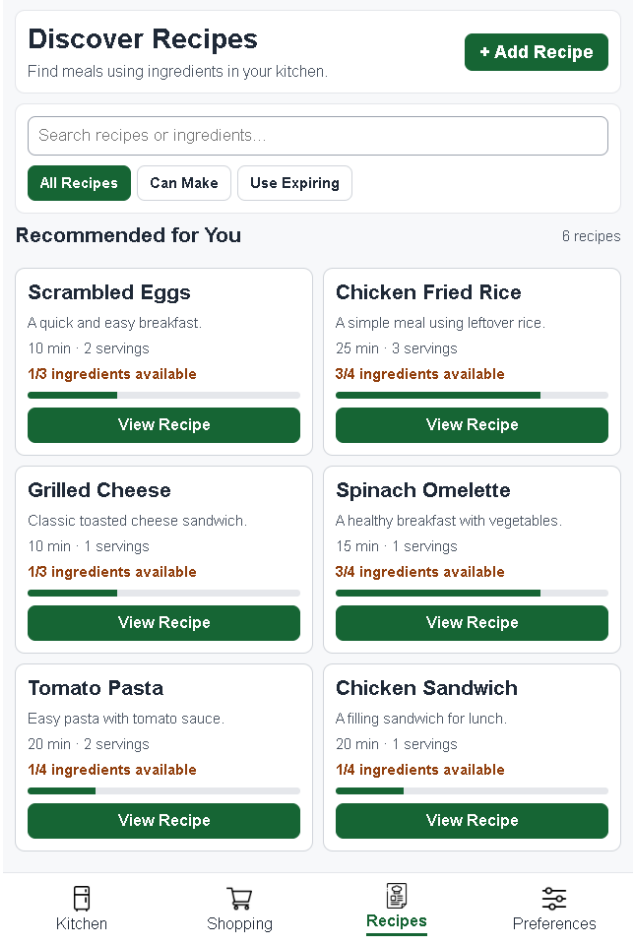
The task suite was exercised with network access disabled in desktop Chrome on the available Windows host at 640 by 960. Checks covered ingredient matching, expired and zero-quantity exclusions, refresh after edits, search, recipe authoring and revision, persistence, shopping entry and saved-list reuse, transfer, location filtering, sorting, and unfinished-state navigation. The existing inventory regression check also passed for saving, cancellation, validation, literal text, quantities, ownership, and reset. The automated run reported no JavaScript page errors.

These checks are development evidence, not a user study, a formal accessibility audit, or the required clean Windows 10 test. The final extracted ZIP is checked separately for standalone file access. The team still needs to compare its app with the course's separate required scenarios and device specifications, review the submitted rationale in its own words, and supply an agreed who-did-what report. This report does not invent individual authorship, prior feedback, task timing, or participant results. Code changes and this draft were prepared with Codex assistance.

H1 Kitchen and Recipes

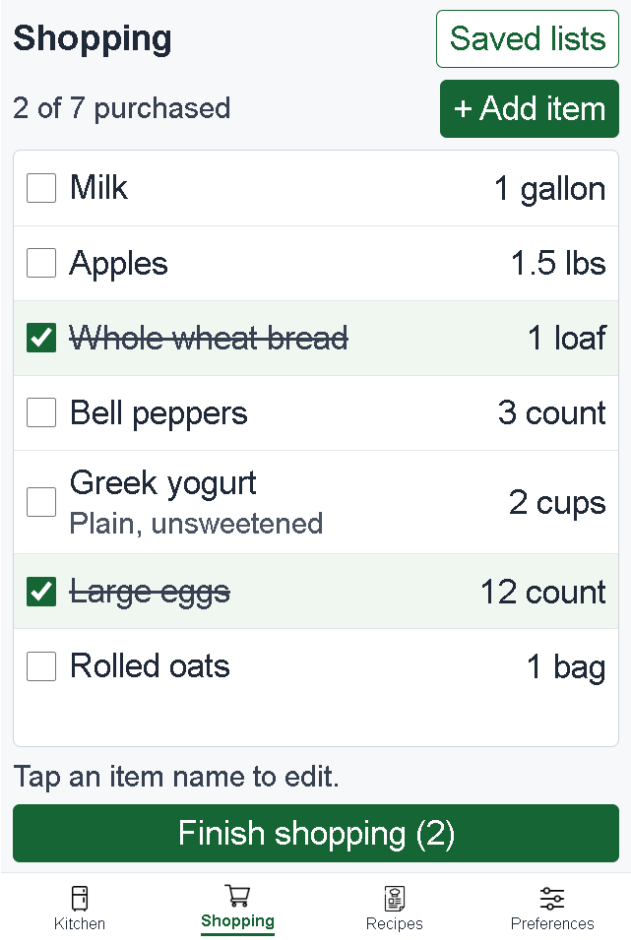


Kitchen opens directly to available food. Search, filters, sort, ownership and quantities are grouped around the inventory.

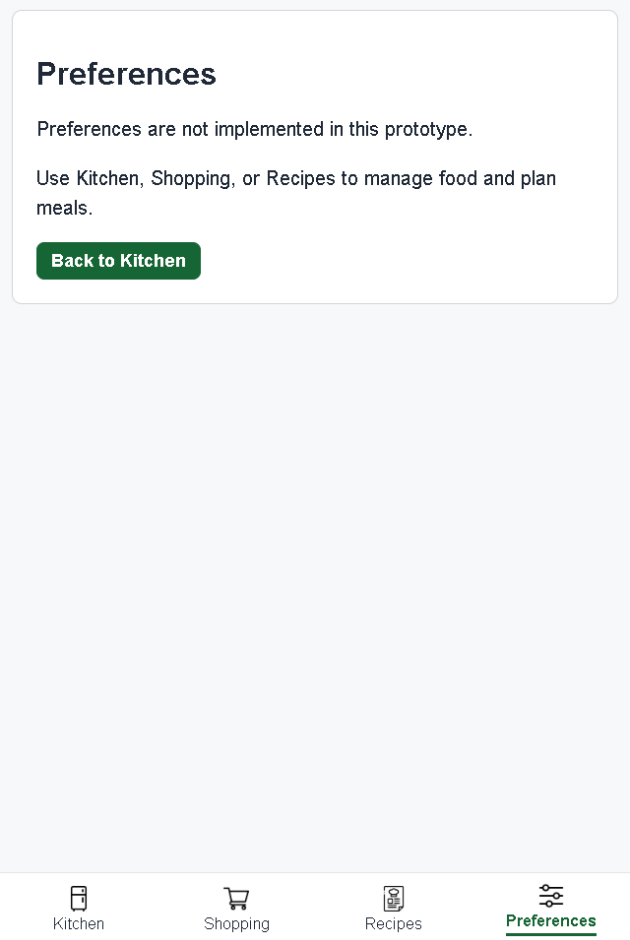


Recipes displays the six sample meals with ingredient counts derived from the actual Kitchen inventory.

H2 Shopping and unfinished Preferences

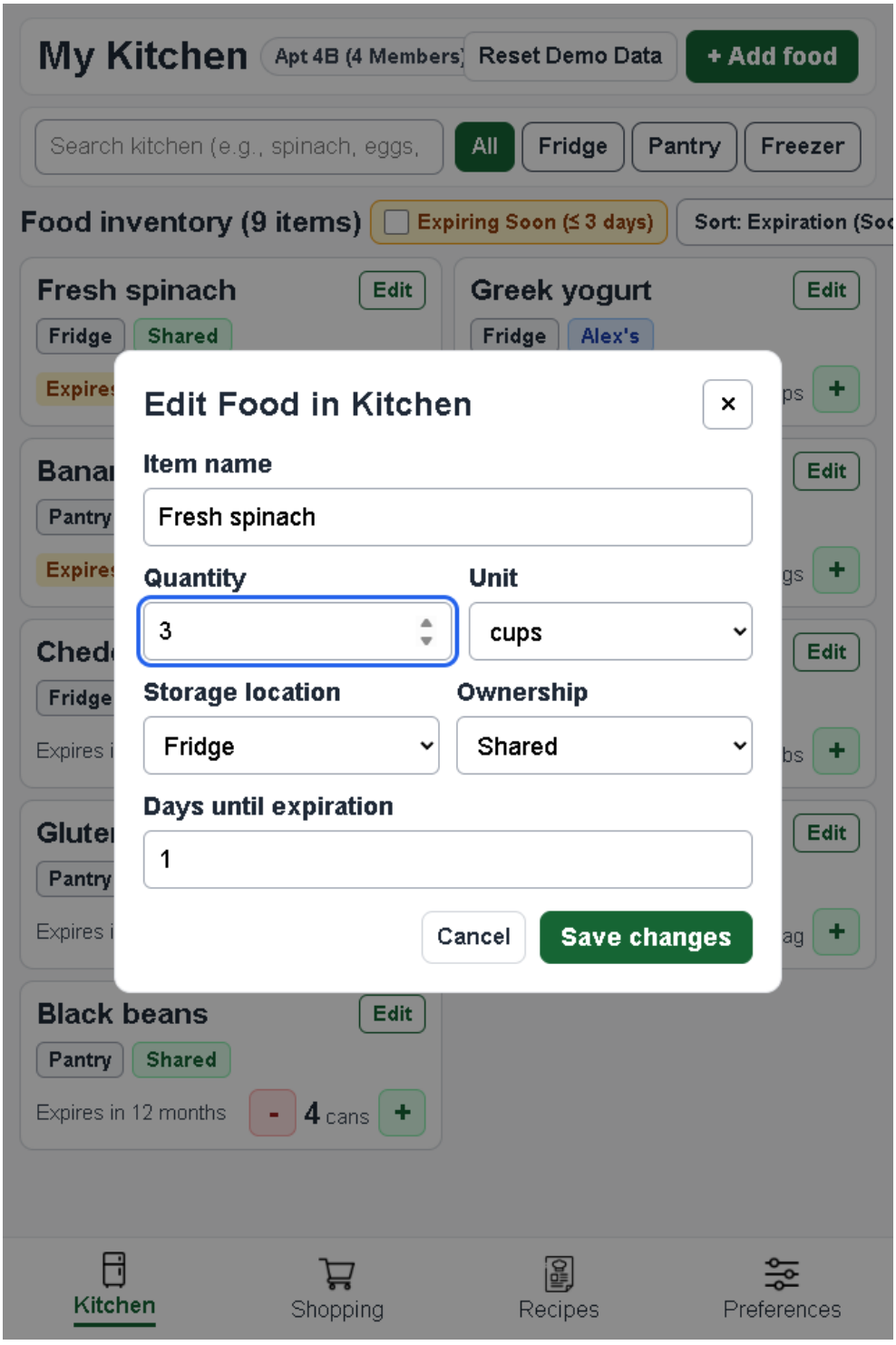


Shopping uses a list with purchase checkmarks, a running count and a distinct Finish shopping action.



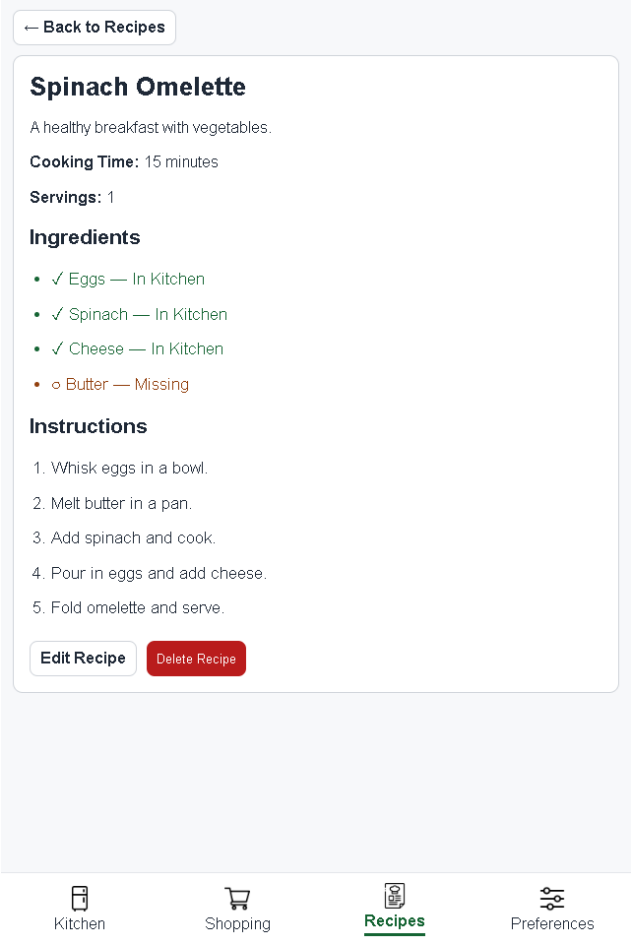
Preferences explicitly identifies its unfinished state and offers a return to Kitchen. It is not counted as an implemented task.

V1 Inventory editing

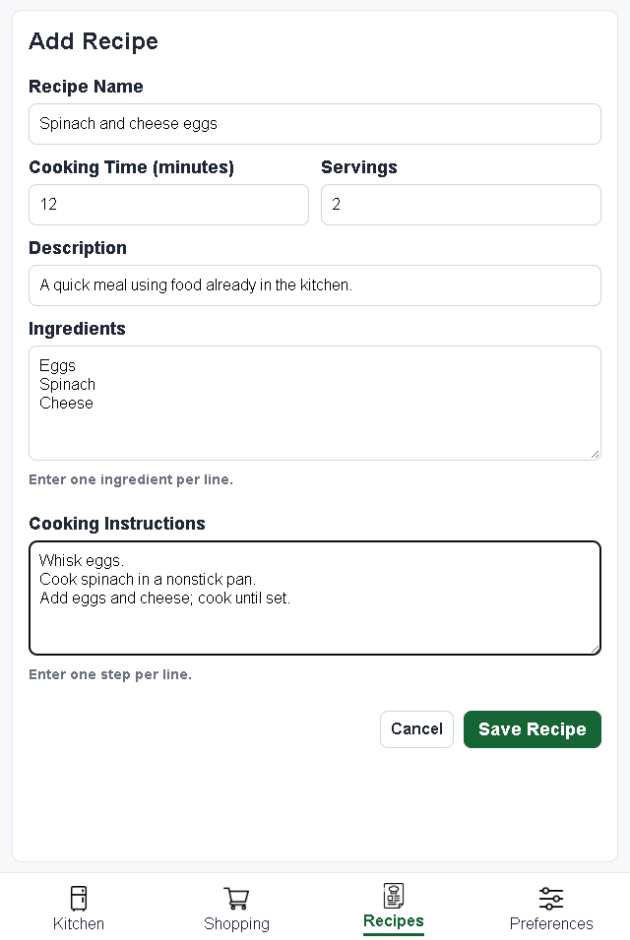


Fresh spinach is opened in the original centered, prefilled editor. Save commits changes to the existing item; Cancel discards unsaved changes.

V2 Inspect and author recipes

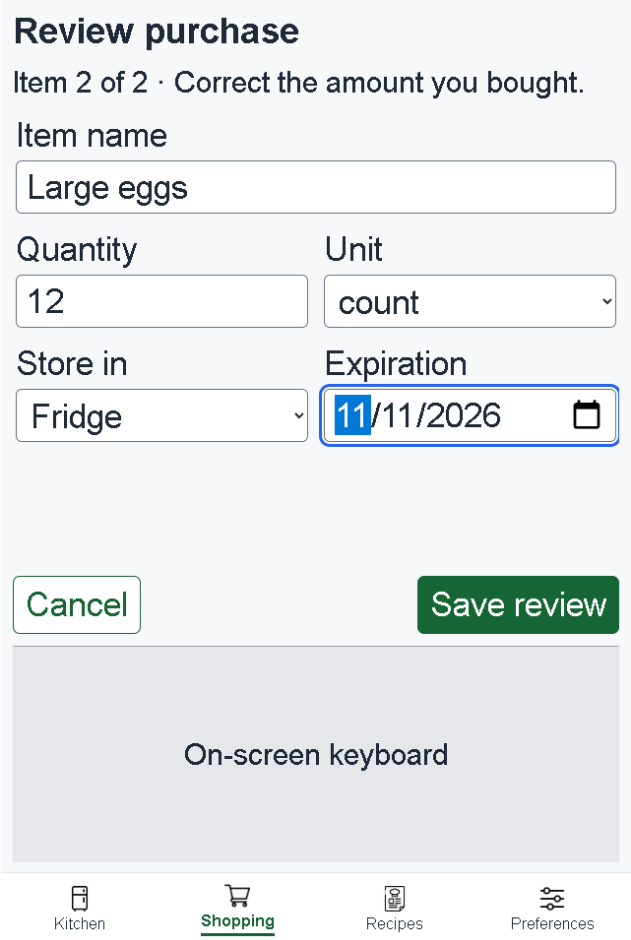


Spinach Omelette recognizes Eggs, Spinach and Cheese from demo inventory aliases; Butter is shown as missing.

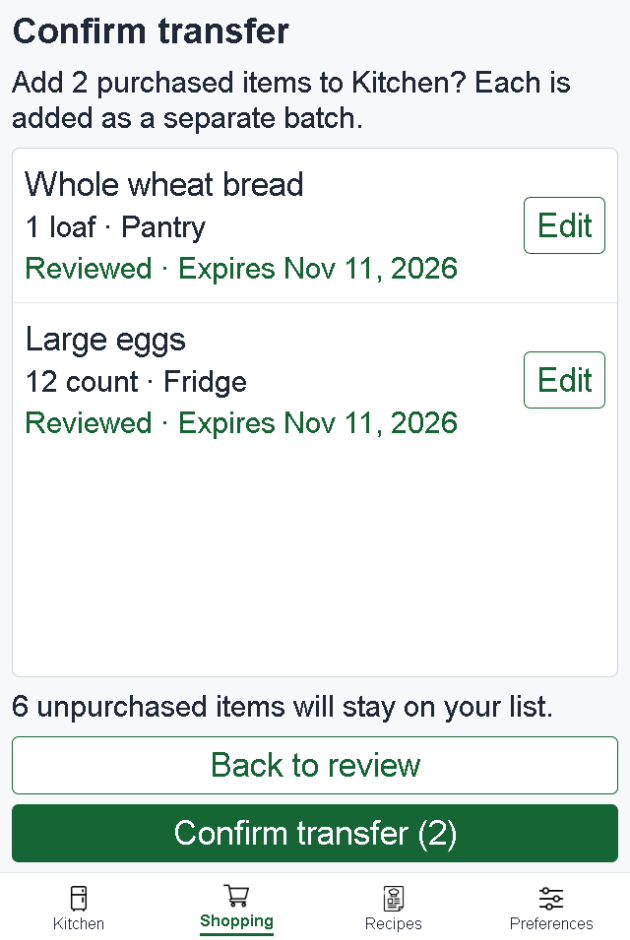


A complete custom recipe is entered with separate ingredients and instruction steps using the original scrolling form.

V3 Review purchases before transfer

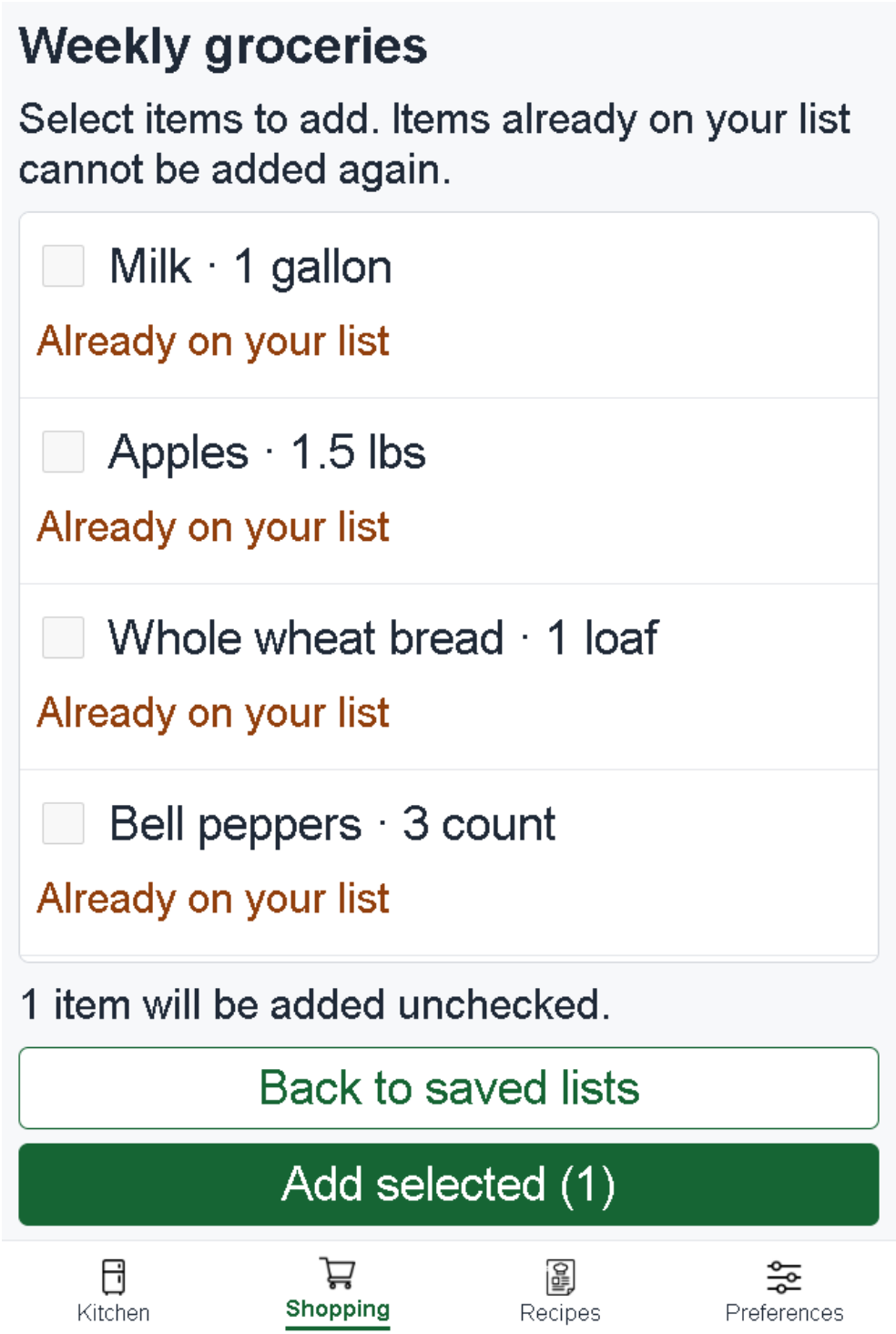


The purchased eggs receive a storage location and date before commitment. The example date is seven days after capture.



The final review presents both batches and explains that unpurchased entries will remain in Shopping.

V4 Selective reuse of a saved list



The named snapshot keeps the saved items. Existing active-list entries are disabled; Butter is the single selected item available to restore. The list can scroll to reveal further entries.